Donghyung Lee

DevOps Engineer | Career Description

GitHub: github.com/somaz94 Blog(EN): somaz.blog Blog(KR): somaz.tistory.com Portfolio: somaz.blog/portfolio

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- **EKS / GKE production cost optimization** – 20% reduction via hybrid transition, 50% via GCP migration — re-platforming Fargate onto GKE node pools plus an MSP reseller discount
- **Solo greenfield EKS build & launch** – the AWS production environment for a new externally-published game — platform components 100% declarative via GitOps, keyless authentication via IRSA / Pod Identity, soft launch opened 2026.07
- **IaC standardization** – codified on-premises, AWS and GCP with Terraform and Ansible — 100% IaC for new components, 100% Git-traceable change history, the same pipeline rebuildable in 30 minutes
- **GPU infrastructure for AI workloads** – a two-card A100 40GB training server on GCP built as Terraform IaC (NerdyStar), and an in-house LLM inference service on an on-prem GPU box (Phantom)

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- **Pipeline design & hardening** – designed multi-cluster CI/CD pipelines on GitHub Actions, GitLab CI, Jenkins and ArgoCD with the deployment automation around them
- **Deploy productivity & reliability** – 67% deployment time reduction with continuous GitOps deployment shortening time to market
- **Canary zero-downtime rollout** – moved the production game service from Blue-Green to Argo Rollouts Canary, removing the 503 race at promotion
- **CI toolchain standardization** – a layered shared toolchain image plus an automated version cycle cut CI build-environment provisioning from 5min to ~10s (~95%), with 100% tool-version consistency

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- **Declarative logging modernization** – ELK → EFK → ECK Operator-based Elastic Stack 9.x CRD declarative management (automatic TLS rotation, automated rolling upgrades)
- **Metrics monitoring** – kube-prometheus-stack with custom alert rules and Grafana dashboards, plus multi-DB exporters (MySQL, Redis, Elasticsearch, PostgreSQL) folded into one surface
- **Tracing cost & accuracy** – wired OpenTelemetry + Tempo to complete the Metrics ↔ Traces ↔ Logs picture, splitting the sampling point to keep metric accuracy while cutting trace storage cost

Automation & Team Enablement

Operations automation tooling with Python and Bash, plus training and standardized materials that let client ops teams run things themselves

- **Ops toolchain automation** – migrated shell → Python (stdlib only) and built the wave-sequenced auto-deploy plus auto-upgrade MR pipeline
- **Engineering enablement** – designed and ran a 6-month hands-on Kubernetes program for the DP (Delivery Partner) ops team (avg 10 attendees, 4h/session), with the materials standardized for reuse
- **Open engineering, continued** – a pipeline that collects, quality-checks and republishes financial and economic data from 10+ public APIs, running twice daily on weekdays (economy dashboard), alongside a backend-free diagram editor run in the open

Kubernetes Platform Modernization & Open-Source Contribution

Network/logging stack migration, public charts released as open source, and upstream contribution

- **Gateway API migration** – migrated multiple ingress-nginx instances onto a single NGINX Gateway Fabric control-plane (incident-free full cutover, wildcard TLS consolidated)
- **Open-source shared assets** – released a number of open-source Helm charts alongside Kubernetes CRD controllers, composite GitHub Actions and Ansible collections on ArtifactHub / GitHub Marketplace / Ansible Galaxy
- **CNCF upstream contribution** – Gateway API HTTPRoute support to Helm charts in apache/airflow, OpenTelemetry and prometheus-community, plus bug fixes to Go projects

Infrastructure Security & IAM

Centralized in-house authentication and secret governance via Keycloak Operator SSO and Vaultwarden

- **Central authentication (SSO/OIDC)** – Keycloak Operator brought single sign-on onto existing GitLab accounts, with ArgoCD / Harbor / Vaultwarden OIDC integrated and verified
- **In-house secret governance** – deployed Vaultwarden on Kubernetes via Helm with GitLab SSO, automated backups and an interactive restore script — central management and a recovery path
- **GitOps secrets** – replaced plaintext DB passwords in Git with External Secrets Operator wired to AWS Secrets Manager, standardizing the Harbor pull secret on Sealed Secrets

Professional Experience

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As the sole DevOps infrastructure engineer at a game development company, I design, build and operate a hybrid architecture — dev and qa on-premises, review and prod on AWS — that cut monthly infrastructure cost by 20%.
For the live-service game I provide second-line technical support and operations with the external publisher, and I am building on-premises/AWS infrastructure to raise development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution: hybrid architecture design & build 100% (solo) / second-line technical support and operations for the live game in collaboration with the external publisher's ops team 2024.10 ~ Present (In Operation)

Problem

The entire infrastructure ran in a single AWS environment at a high monthly cloud cost.
Non-production environments, whose traffic barely varies, were holding cloud resources around the clock — the estate needed both cost optimization and environment separation.

Approach

- **Hybrid architecture design** – analyzed cost and usage patterns per workload, then moved dev·qa on-premises and kept review·prod, the environments that need scale and high availability, on AWS
- **Network and pipeline isolation** – separated traffic at the application layer rather than standing up a site-to-site VPN, and split the image registry (dev·qa = Harbor, review·prod = ECR) so each environment builds and operates independently
- **IaC-based operational standardization** – introduced Terraform and Ansible across both on-premises and AWS, reaching 100% IaC for new components and establishing one configuration-management standard across heterogeneous infrastructure
- **IAM credential governance** – only service and CI accounts are managed as code, with console accounts kept separate.
  Terraform never generates user access keys — they are issued and rotated individually — so no long-lived credential is left in plaintext in the Terraform state file

Troubleshooting

- **Version-aware backend routing for game clients** – a single ALB Ingress on EKS had to route old-client and new-client requests to different backends.
  Combined ALB conditional routing rules on a custom HTTP header with target-group mapping, so the split needed no additional infrastructure
- **Stopped L7 connections dropping during rolling deploys** – the ALB target deregistration delay and SIGTERM timing were misaligned, so new requests reached a terminating Pod while in-flight requests were cut. A preStop hook fails the health check first, blocking new arrivals while the remaining traffic drains within the deregistration delay; RollingUpdate was tuned add-before-remove to hold capacity through the rollout — zero connection drops.
  Verified on a CloudWatch-backed Grafana dashboard reading ELB 5xx and Target 5xx separately across each deploy: zero 5xx, p95 held

Results

- Cut monthly infrastructure cost 20% by re-placing workloads on the analysis, with the saving reinvested into new project infrastructure
- Physically isolated dev·QA from production, containing the blast radius of any failure at its source and securing operational stability

Project 2: Game Application CI/CD Pipeline & Shared Toolchain Standardization (App GitOps)

Contribution 100% (Sole design and build across the pipeline and CI standardization) 2024.12 ~ Present (In Operation)

Automated the game application's manual build-and-deploy into a GitOps pipeline on GitLab CI and ArgoCD, then standardized the CI toolchain image layers and version governance shared by every in-house repository on top of it, bringing build-environment provisioning and tool-version management into a single flow.

2-1. Game Application Deployment CI/CD Pipeline (App GitOps, 2024.12 ~ Present)

Problem

Developers built and deployed to the servers by hand — about 30 minutes per deploy, with human error causing frequent incidents.

Approach

- **GitOps continuous delivery pipeline** - GitLab CI wired to ArgoCD so a Git push drives build → test → deploy end to end, with ArgoCD detecting any mismatch between the configuration declared in Git and the cluster state and self-healing back to it; the deployment logic for all four environments — dev, qa, review and prod — was unified into that single pipeline, so they are managed consistently
- **Least-privilege image build environment** - to avoid granting the host administrator privilege (privileged mode) that Docker-in-Docker (DinD) requires, image builds moved to Kaniko, which builds container images in user space without a Docker daemon — closing off the CI runner's security exposure and the risk of privilege takeover at the source

Troubleshooting

- **Preventing ArgoCD cascading prune** - to stop a Git commit error from cascading into the deletion of every child Application, workload Applications are reconciled automatically with `prune: true` + `selfHeal: true`, while the top-level app-of-apps root and the AppProject layer keep `prune: false`, so resource deletion is gated behind manual verification.
  Every sync event is tracked in real time through Slack notifications
- **Removing static SSH keys from the data deployment path** - the CI deploy account held an SSH key at all times and connected straight to the target instances (scp/rsync); that path was replaced by uploading artifacts to an S3 transit bucket and having each instance pull them itself via SSM SendCommand — static SSH keys are eliminated from the deployment environment, closing off the credential leakage risk

Results

- 67% shorter deployment lead time (30min → 10min) — the manual build, deploy and configuration steps were unified into the pipeline, eliminating human error as a root cause of deployment incidents
- ~78% shorter source fetch time in the data deployment (~139s → 30s) — a Git optimization that fetches only the latest commit and only the files actually needed (shallow + blobless fetch) removed the transfer bottleneck
- GitOps-based continuous delivery established as routine — a steady ~100 deploys a week per core service repository, and a shorter time for a new feature to reach production
- Built a multi-repository MasterData pipeline — a design-data change is propagated to the downstream services automatically and in one batch, with runaway repeated execution of the pipeline structurally prevented

---

2-2. Shared Toolchain Image Layers + Automated Version Cycle (2025.09 ~ Present)

Problem

In every repo in the GitLab CI standardization scope, each job re-installed helm / helmfile / kubectl / crane / aws-cli on an Alpine base, so provisioning averaged ~5 minutes; tool versions were scattered across consumer repos, so adding one meant touching every repo.
Reliance on `:latest` also undermined reproducibility and CVE tracking.

Approach

- **Layer 1 (Mirror)** - copied external registry images (Docker Hub, GHCR) into the internal Harbor via `crane copy`, preserving multi-arch manifests.
- **Layer 2 (Custom)** - built fat images on top of Layer 1 (helmfile-tools, base-tools, node-build, rclone-backup, aws-cli-tools) so consumer repos get the tools immediately on image pin alone.
- **Tier 1~2 SSOT** - introduced a shared `.toolchain_build_custom` template + central `TOOLCHAIN_*_TAG` variables so consumer repos no longer track image tags directly.
- **Shared CI template SSOT** - extracted the per-repo copy-pasted CI jobs into a shared template repo referenced via `include`; the auth (keyless AWS auth) / build (kaniko) / deploy (ArgoCD) / backup (Google Drive) job templates plus shared anchors (git·ssh·rules·job-config·packages) live in one place, eliminating copy-paste drift.
- **Automated version cycle** - a weekly schedule runs (1); once an operator starts the pipeline, (2)-(4) follow as auto-MRs. Human judgment sits at exactly two points — starting the pipeline and merging the Tier 2 MR:
  · (1) Upstream version detection
  · (2) Layer 1 mirror auto-MR
  · (3) Layer 2 rebuild + Tier 2 TOOLCHAIN_*_TAG sync auto-MR
  · (4) Auto-propagation to the consumer repos (excluding the template provider repo)

Troubleshooting

- During auto version bumps, helmfile 1.5.1 began requiring helm >= 3.18.6 while helm stayed pinned at 3.16.4, breaking the `build_custom` build.
  Since cross-tool compatibility surfaces only in upstream release notes and cannot be auto-detected, introduced a minor-guard for helm / helmfile / kubectl: only patches within the current minor bump automatically, while minor/major bumps wait for a human compatibility check (overridable once via `*_TARGET_MINOR` env vars for a planned minor bump)

Results

- **Build-environment provisioning cut from 5min to ~10s (~95% reduction)**, with 100% tool-version consistency across the consumer repos, compressing the scattered version points into a 4-tier governance flow (ARG → central var → consumer indirection → lint drift check)
- Automated new-version propagation for helm / helmfile / kubectl / crane / rclone, with the high-blast-radius image rebuild gated behind human review rather than the schedule
- Built a risk-tiered auto-upgrade pipeline across components (T1 weekly / T2 biweekly / T3 monthly + pre-flight) — version detection → auto-MR → Slack alert → wave-sequenced apply, with a MAJOR_PIN guard and T3 atomic rollback — and built amd64 + arm64 multi-arch images via docker buildx, with the Layer 1 mirror removing external-registry dependency

---

Project 3: Central Logging & Observability Platform (Collection → Integrity/HA → Product Analytics)

Contribution 100% (Sole design across the full lifecycle — collection · integrity · product analytics) 2024.11 ~ Present

In a log-scattered on-premises environment, advanced centralized logging through three stages, hardened data integrity and high availability (HA) across the log collection layer, then extended the accumulated EFK pipeline into a game user-retention analytics platform.

3-1. Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement, 2024.11 ~ Present)

Problem

Server and container logs were distributed across the on-premises environment, making root cause analysis time-consuming during incidents, with no unified monitoring system in place.

Approach

- **ELK → EFK** - per-node Filebeat collection into a central Logstash tier, whose JVM-based processing grew memory-hungry → collection by a lightweight Fluent-bit DaemonSet, parsing by Fluentd
- **Operator-managed** - Elastic's official Helm chart stalled for over two years → CRD-based declarative management under the **ECK Operator**
- **Migration to AWS(EKS) and automation** - templated the on-premises EFK stack for AWS(EKS) and deployed it there; log lifecycle (ILM) and S3 snapshot storage are automated through IRSA with no static keys

Troubleshooting

- Auto-updating Elastic Stack releases on the on-premises stack referenced an image registered in the release feed but never actually published to the registry → `ImagePullBackOff`. An immediate rollback was attempted, but the ECK admission webhook's downgrade-refusal policy severed the recovery path and the failure spread to the operator.
  Defining the inability to roll back — rather than the deployment failure itself — as the core operational risk, the upgrade script was systematized with registry existence verification for the image, a cluster health pre-flight check, a snapshot-backup warning on major version bumps, and a safe recovery procedure accounting for the webhook constraint, preventing recurrence

Results

- Central aggregation for logs from distributed nodes and ephemeral pods — log loss when a pod is recreated or terminated is eliminated at the source, and time-series tracing in a single Kibana view sharply cut the time to pin down the cause of an incident (MTTR)
- Management automated and brought current through the ECK Operator migration — the manual burden of TLS certificate renewal, account provisioning and node replacement is gone (declaring the version in the CRD is enough to guarantee a rolling upgrade), and the Elastic Stack major upgrade stalled for over two years was completed (8.x → 9.x)

3-2. Fluent-bit Log Collector Data Integrity & HA Hardening (2026.03 ~ Present)

The SQLite offset DB is kept on node-local storage (`hostPath`) so the read position survives a pod restart (zero re-index, zero duplicates achieved), and monitoring alerts are wired to detect the abnormal stall state in which log collection halts without the process going down, preventing the log loss that would otherwise go unnoticed

---

3-3. Game User Retention Analytics Platform (EFK Logs → Product Analytics, 2026.06 ~ Present)

Extended a search-only EFK stack into a product-analytics platform via an Elasticsearch Transform (5-min) pivoting events to one row per user — a Kibana dashboard delivering D+1~D+30 retention and cohorts, no separate SaaS

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server)

Contribution 100% (Full system design & development) 2025.02 ~ Present

Designed, developed, and operated 5 internal tools to automate repetitive manual tasks and boost development team productivity.

4-1. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Deployed a Python bot on Kubernetes auto-sending completion messages and download QR codes to Slack on Jenkins APK build finish — 90% QA delivery-time reduction, version confusion eliminated

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.06)

One-way sync of design documents from GitLab into a Google Drive shared drive via rclone, triggered on push — comparing on `--checksum` so only changed files transfer, with a sparse partial clone of just the changed directory cutting how much of the large repository moves — 80% doc-management reduction (10min→2min), GitLab held as the single source of truth with direct Drive edits overwritten on the next sync

4-3. Internal LLM Service Deployment (4 weeks, 2025.11)

Ran an in-house LLM on an on-prem GPU server with Ollama and Open WebUI on docker compose (used by the dev team)

4-4. Git Repository Mirroring Tool + Retry API Follow-up (4-week initial build, 2026.02 ~ Present)

Built and open-sourced git-bridge, a Go mirroring tool, to resolve the gaps and lag of manual syncing between CodeCommit, GitLab and GitHub — 100% automation of ~30 syncs a day across 10 repositories, with failure retry and Slack notifications making sync status trackable in real time

4-5. Static File Server - In-house Development (Open Source, 2026.03 ~ Present)

To replace an existing file server that had stopped being maintained and lacked the core features — directory UI, search, ZIP batch download — built a Go static file server in-house and released it as open source, deployed to the cluster with an in-house Helm chart wired to NFS storage as a full replacement for the previous tool, reliably handling ~30 downloads a day and an APK release pipeline that runs 5 times a week

Project 5: Kubernetes Cluster Operations Enhancement

Contribution 100% (Solo design — standard procedures captured as automation scripts the team reuses) 2025.12 ~ Present

To keep the cluster running reliably, overhauled monitoring, automated K8s upgrades and Helm management, moved the network controller onto the Gateway API, and went on to resolve storage bottlenecks and modernize the deploy/upgrade pipeline.

5-1. Monitoring & Alerting Enhancement (2025.12 ~ Present)

Designed custom alerts and Grafana dashboards on kube-prometheus-stack, integrating DB, node-exporter, and Cilium metrics — custom alert rules grouped by component, with severity routing and inhibit rules removing duplicates

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ Present)

Scripted the Kubespray upgrade procedure and a nine-item post-upgrade check (nodes, etcd, API server, certificate expiry and more), standardizing version management with a Helm framework and a canonical-template synchronizer — 90% verification-time reduction, with certificate renewal scripted to remove expiration outages

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Migrated the on-prem ingress-nginx fleet to a single NGF 2.x control-plane + per-class Gateway CRs — MetalLB IPs preserved so DNS/firewall unchanged, incident-free cutover (30-60s per class), one wildcard cert cutting manual renewal 50%

5-4. Storage Performance Optimization — etcd / DB SSD Migration + Build I/O Isolation (2 weeks, 2026.04 ~ 2026.05)

Diagnosed the bottleneck in which GitLab Runner (buildx) disk-I/O spikes drove second-scale latency on HDD-backed etcd and the game DB, migrated the control plane and the DB worker to SSD, and fully isolated the build workload onto a dedicated VM on separate physical hardware — game DB COMMIT latency normalized from 9~14s to the millisecond range, with zero incidents of blocked in-game connections or service latency during builds

5-5. Kubernetes Infrastructure Deployment Automation & Multi-cluster GitOps (ArgoCD) Migration (2026.03 ~ Present)

Problem

The Shell/awk-based infrastructure monorepo could not be test-automated — complex YAML manipulation and per-environment branching had outgrown it — and CI-runner-centred push deployment (`helmfile apply`) could not detect cluster state drift and did not scale to multiple clusters.

Approach

- **Tooling refactored onto the Python standard library**: Python3 modules that take no third-party package, with a 1:1 module-to-test `unittest` suite, giving identical execution on a local machine and on a CI runner
- **Wave-based deployment and upgrade automation**: a six-stage wave order derived from component dependencies, plus a pipeline that detects a new chart version and opens an MR carrying the values diff and the compatibility checks

- **Moved onto an ArgoCD pull-GitOps control plane**: ApplicationSet (Matrix / Git generators) automates deployment across the on-prem and AWS clusters, and existing Helm releases were adopted into ArgoCD management without recreating a single resource and without an interruption
- **GitOps governance and safety rails**: cascading prune blocked at the top layer plus an AppProject whitelist, isolating the blast radius

Results

- Platform releases across the on-prem and AWS clusters are controlled declaratively from a single GitOps control plane
- The legacy shell scripts were migrated wholesale to Python tooling backed by unit tests, making deployment dependable
- Automated chart-upgrade MRs and wave deployment together cut the operational overhead of running the infrastructure sharply
- Declarative self-healing of the desired state and per-layer prune control eliminated the risk of a large-scale deletion incident at the source

---

Project 6: Infrastructure Security & Governance

Contribution 100% (Design, deployment, SSO integration) 2026.04 ~ Present

Standardized the fragmented account authentication, secret management and remote-access paths into one internal security governance model.

6-1. Central Authentication (SSO/OIDC) on Keycloak Operator (2026.04 ~ Present)

Consolidated the GitLab OIDC paths — wired separately into each in-house tool (Harbor, ArgoCD, Vaultwarden) — onto a central IdP on the Keycloak Operator.
The permission flow GitLab group → Keycloak group mapper → ArgoCD role now runs through one place, giving central control of the authentication policy and a single sign-on for every tool

---

6-2. Secret Governance — Vaultwarden · GitOps Secrets (2026.04 ~ Present)

Moved the shared internal passwords scattered across mail and messengers onto Vaultwarden on Kubernetes, wired to SSO, establishing one place to manage them.
DB credentials that had been committed to Git in plaintext moved onto External Secrets Operator backed by AWS Secrets Manager, and the Harbor cluster secret was standardized on Sealed Secrets — closing off credential exposure in the GitOps repository at the source

---

6-3. Secure Remote Access to the Internal Network (OpenVPN PKI, 2026.07)

Built an on-premises VPN gateway for the internal network sized for 50 concurrent connections.
Per-user EC certificates (prime256v1) issued from an easy-rsa PKI plus tls-crypt encryption of the control channel (AES-256-GCM, TLS 1.2+) turn away an unauthorized attempt at the handshake, and issuing and revoking a certificate is automated end to end by idempotent shell scripts

---

Project 7: AWS EKS Production Game-Launch Platform (New Externally-Published Title)

Contribution 100% (solo design, build & operation) 2026.06 ~ Present (Building & Operating)

Solely built a greenfield EKS production environment in eu-central-1 for the commercial launch of a new externally-published game (7-1), then hardened it with distributed tracing (7-2) and Canary zero-downtime delivery (7-3) through to the soft launch (7-4).

7-1. Greenfield EKS Production Infrastructure & Live-traffic Tuning (eu-central-1, 2026.06 ~ Present)

Problem

A single on-premises cluster had reached its limits for global traffic and flexible autoscaling, so an independent, highly available EKS production architecture was needed for the overseas commercial launch.

Approach

- **Hybrid multi-cluster, declaratively managed**: stood up a greenfield EKS cluster in eu-central-1 and brought every platform component under one declarative GitOps (ArgoCD) model
- **Compute separated and tuned on Karpenter**: the game services sit on on-demand nodes for stability, while CI runners spread across a diversified ARM multi-family spot pool, cutting cost and minimising reclaim
- **Networking and authentication automated**: AWS Load Balancer Controller, ExternalDNS and a wildcard ACM certificate together automate ingress and TLS issuance end to end, and every ServiceAccount runs on Pod Identity / IRSA, removing static IAM keys
- **Logging and secret foundation**: an eck-operator-based HA EFK stack (3 nodes across 3 AZs, EBS gp3, S3 snapshot SLM) plus External Secrets Operator gave the new cluster its own log collection and secret-reference path
- **Credentials isolated out of the deploy path**: client artifacts ship Jenkins → S3 → CloudFront, and server manifests reach the bastion's EFS mount through an S3 transit bucket and SSM SendCommand, so no SSH key is held anywhere in the path

Troubleshooting

- **Node scale-down and live game sessions reconciled**: blocking reclaim outright (`karpenter.sh/do-not-disrupt`) would have left idle nodes billing, so it was ruled out; with sessions externalised to Redis the services are stateless, and preStop drain + PDB (`minAvailable: 1`) + a disruption budget of `nodes: 1` retire one node at a time safely
- **Idle nodes never reclaimed after a rolling deploy**: diagnosed an over-sized memory request (2 GiB against a measured 101 MiB) as the cause of lopsided node occupancy (92% against 35%) and of consolidation never firing.
  Bringing the request down to 512 MiB and adjusting the consolidation condition restored bin-packing density and automatic reclaim
- **Traffic reaching a pod that was still starting**: intermittent 5xx errors from the ALB forwarding to a pod mid-startup were resolved with an ALB pod readiness gate — a pod counts as ready only once it has passed the target-group health check

Results

- An independent AWS hybrid multi-cluster for the overseas launch, with 100% of the platform manifests under declarative GitOps management
- Static credentials removed across the infrastructure, the bastion SSH key included
- Production and review workloads share one NodePool, removing the minimum one node each separate pool would have held and maximising bin-packing density
- The GitOps manifests and the CI/CD migration procedure are documented as a standard runbook, so the work can be handed over and the same environment rebuilt

---

7-2. OpenTelemetry End-to-end Distributed Tracing Pipeline (OTel + Tempo, 2026.06 ~ Present)

Problem

The existing observability stack — EFK for logs, Prometheus for metrics — gave only a limited view of how calls flow between microservices, where the latency bottleneck sits, and how an error propagates across a distributed environment.

Approach

- **Tracing stack with non-invasive instrumentation**: an OpenTelemetry Collector and Grafana Tempo (S3 backend, wired through EKS Pod Identity with no static key) on AWS EKS, with the OTel Operator's auto-instrumentation injecting the per-language SDK without touching application code
- **Sampling split for observability accuracy and storage cost**: RED metrics are derived from 100% of spans before sampling, keeping the aggregates exact, while storage goes through tail sampling — every error, every request over 500ms, and 10% of normal requests — to cut storage cost

Results

- **3-signal cross-linking**: metrics, traces and logs are linked so one Grafana view moves between them, making the call flow between services and its bottlenecks visible (the existing logging is Project 3, metrics 5-1)
- **Two-layer routing to fix trace fragmentation**: identified a structural flaw in the multi-replica collector — each app pod's gRPC connection is pinned to a different instance, so a trace that crosses pod boundaries can have its spans scattered, which can break both keep-every-error-trace and the service-graph join
- Split the pipeline into a receiving layer (layer 1) and a sampling layer (layer 2), routing by trace ID through the loadbalancing exporter over a headless Service, so every span of a trace always reaches the same sampling instance
- Validated on live traffic: the per-replica share, skewed by connection pinning to 1,650 : 541 in series count, evened out after trace-ID hashing to 4.09 : 5.18 spans/s
- **Closing the sampling-layer alert gap**: after the split, the existing collector-down alert could not see a full outage of the sampling layer; four alerts were added — sampling layer down, traces dropped undecided on buffer overflow, buffer near full, and policy evaluation errors — to watch for silent trace loss

---

7-3. Zero-downtime Delivery on Argo Rollouts (Blue-Green → Canary, 2026.06 ~ Present)

Problem

Blue-Green promotion swapped the ALB target group wholesale, leaving a window in which the healthy target count was 0 — and the ALB answered 503 intermittently during it.

Approach

- **Moved to a canary rollout**: within the same ALB target group, `maxUnavailable: 0` plus add-before-remove (the new pod registers before the old one is withdrawn) lets old and new coexist briefly, removing the 503 race
- **Resting on a stateless design**: the game services are stateless HTTP (state externalised to JWT and Redis/TypeORM), so traffic hitting both old and new mid-rollout is safe.
  A change that cannot have both versions live at once — an API contract change, say — falls outside this strategy and is handled in a scheduled maintenance window
- **Entry and exit lifecycles synchronised**: an ALB pod readiness gate admits a pod to the traffic pool only once it can actually serve, and on the way out a deregistration delay of 30s → preStop 35s → terminationGracePeriodSeconds 60s drains in order, so in-flight requests finish — 75s for the .NET service, which drains its own in-flight requests on SIGTERM
- **Strategy templated, controller made HA**: the delivery strategy is a template in the shared Helm chart and canary is applied to the one public-facing production game service (the same game's internal services, which take no public traffic, stay ordinary Deployments), while the controller runs leader-election HA behind a PDB so a rollout survives node churn

Results

- The healthy-target-0 window and the 503 race that came with promoting a production game deploy are gone at the source; a backward-compatible patch now ships with no downtime

---

7-4. Soft-Launch Operation (2026.07 ~ Present)

Results

- Soft launch went live 2026-07-14 (KST) — the limited release's small-scale real traffic handled on the Canary deploys and 3-signal observability built above
- Observability, deployment and autoscaling (HPA · Karpenter) all in place for the 2026 Q4 global launch (no scale-out has fired yet at soft-launch volume)
- Re-sized workload requests and limits from 14 days of measured soft-launch traffic — requests set from the observed per-pod peak so the HPA actually triggers (an over-sized request pins utilization below the HPA target permanently and the scale-out never fires, as it never had under the previous values)

Project 8: Second EKS Cluster on Auto Mode & a Site-to-Site VPN (WireGuard) Gateway

Contribution 100% (solo design & build) 2026.08 ~ Present (Building & Migrating)

Problem

The new publishing title had to reach the publisher's internal authentication account, and that path could not cross the public internet — it needed a private route over VPC peering. The existing production VPC's address range did not meet the peering requirement, which made recreating the VPC and everything above it, the cluster and the data tier included, unavoidable; and the publisher asked for the rebuilt cluster to run on EKS Auto Mode.

The existing cluster ran self-operated Karpenter on managed node groups, so the compute-side Terraform and state management points had grown large and node lifecycle, spot interruption and the core addons were all managed by hand. VPC peering is also non-transitive, so a private path from on-premises to the internal (private) ALB had to be provided separately.

Approach

- **Compute layer moved onto EKS Auto Mode**: node provisioning and lifecycle, the Karpenter satellite resources (IAM, SQS, EventBridge) and the built-in addons (LB controller, EBS CSI) all moved to AWS-managed, leaving only a declarative NodePool and sharply reducing the complexity of the infrastructure code
- **Re-platformed around the IPAM constraint**: with no room to grow the subnets (four /20s exhaust the parent /18 exactly), isolation moved from physically separate subnets to security groups inside a single private subnet, relocating Aurora, Valkey and EFS
- **Site-to-site WireGuard VPN built for a dynamic public IP**: the office's public IP is dynamic, so AWS's managed VPN — which requires a fixed customer-gateway address — was not an option; the tunnel is established outbound from on-prem toward an AWS EIP, and the peer endpoint is deliberately left empty so the first handshake packet follows the address wherever it moves
- **Platform components ported for compatibility**: 16 core platform components — observability, security, the CI toolchain — were reworked for Auto Mode, and 10 of them moved onto the ArgoCD pull-GitOps track

Troubleshooting

- **Bootstrap deadlock from the Auto Mode system taint**: the built-in system NodePool's `CriticalAddonsOnly` taint means capacity for an ordinary pod is zero until your own workload NodePool exists — so ArgoCD itself sat Pending and GitOps could not sync at all.
  The NodePool and IngressClass manifests were pulled out of the GitOps track and moved into a helmfile pre-bootstrap stage, separating the initial ordering
- **No load balancer, and no error either, from a missing IngressClass**: the IngressClass had been provisioned by the AWS Load Balancer Controller Helm chart, and Auto Mode ships the controller built in without that chart — so nothing created it. Even with `ingressClassName: alb` set there was no matching controller, and the load balancer simply never appeared, with nothing in the logs.
  An Auto Mode-specific IngressClass is now its own component, deployed immediately after the cluster is created
- **NodeClass moved API group, and one addon is not included**: a NodePool is still written the same way, but NodeClass moved from `karpenter.k8s.aws/v1` EC2NodeClass to `eks.amazonaws.com/v1` NodeClass, so the existing manifests no longer applied.
  The specs were rewritten against the new CRD, and the EFS CSI driver — which Auto Mode does not include — is deployed separately

Results

- Terraform management points in the compute layer went from 102 to 17, an 83% reduction, and node lifecycle and spot reclaim moved entirely to AWS-managed
- A site-to-site VPN gateway between on-prem and AWS keeps the internal ALB off the public internet behind one internal hostname and one TLS certificate — the alternative, a public ALB per environment, was rejected because every new environment would double the names, certificates and DNS records
- Fail-fast routing that rules out gray failure — the static route lives at a single point on the gateway router rather than per pod or per node, so if the path disappears everything fails at once (kept per node, whether the same service works would depend on where its pod landed)
- The three things the tunnel depends on from outside Terraform — the router's static route, the gateway config file, and the private key — are named in the runbook so a dependency invisible to `terraform plan` is at least written down, and the private key is held in SSM Parameter Store as a SecureString rather than sitting in plaintext in a state bucket several people can read

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game/blockchain startup, leading the large-scale AWS → GCP cloud migration.
Drove the transition to capture cost optimization and GCP's global network performance, completing the full service migration within a scheduled game-maintenance window with minimal downtime. Maintained dual source management — GitLab for on-premises, GitHub for production.
Also built a data-collection pipeline consolidating multi-source data (MongoDB · CloudSQL · GA · Dune) into BigQuery with auto-refreshed Google Sheets, eliminating the 2-3 hours of analysts' daily manual work.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Build-out

Contribution: Infra architecture design, migration strategy & execution lead 70% (remaining 30% = server-team application-side migration collaboration), CI/CD pipeline construction 100% 10 months (2023.03 ~ 2023.12)

Problem

AWS costs kept rising, driven largely by per-Pod billing for the always-on game workloads on Fargate.
The migration targeted cost reduction alongside the global load balancing the multi-region game service needed.

Approach

- **Three-phase sequential migration** - Phase 1 built and validated the dev environment on GCP, Phase 2 QA, Phase 3 production
- **Network and authentication** - network designed on Shared VPC (Host Project / Service Project separation); GitHub Actions' GCP auth switched to Workload Identity Federation, replacing service-account key issuance and rotation with short-lived OIDC tokens.
  DNS pre-validated by subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS), leaving only the final NS swap to cut over inside an announced maintenance window
- **Compute re-platforming** - AWS Fargate workloads moved onto GKE standard node pools; per-Pod billing loses to per-VM billing for always-on workloads, so GKE Autopilot — the same billing model — was ruled out
- **Storage minimum bypassed, WAF** - worked around Filestore's 2.5TB SSD minimum by self-hosting an NFS server on Compute Engine (pd-balanced 1TB disk), plus Cloud Armor region blocking and an IP allow-list WAF policy
- **IaC and GitOps pipeline** - codified GCP infrastructure with Terraform (100% excluding IAM), then after the migration built a GitLab CI + ArgoCD GitOps CI/CD pipeline standardizing per-environment config with Helm Charts

Troubleshooting

- While migrating AWS ALB-based routing to a GCP Global LB, traffic-routing issues arose because AWS ALB fails open and still passes traffic when every target is unhealthy, whereas GCP LB blocks it.
  Analyzed the GCP NEG (Network Endpoint Group) structure and reconfigured health checks/backends for GKE workloads
- GKE Ingress + BackendConfig health check returned 503 when the configured requestPath had no response.
  Resolved by guaranteeing the health-check endpoint response and aligning ManagedCertificate / FrontendConfig to restore external ingress traffic
- Game client file downloads failed after the Cloud CDN migration due to a missing HTTP → HTTPS redirect.
  Resolved by splitting the URL Map into HTTP / HTTPS variants (`default_url_redirect.https_redirect=true`) and mapping the HTTP forwarding rule to a dedicated port-80 target HTTP proxy
- After delegating DNS to Cloud DNS, the domain was not auto-registered in Google Search Console, breaking search visibility and ownership verification.
  Added the Search Console verification TXT record to Cloud DNS, verified ownership, and re-registered

Results

- 50% cloud cost reduction — designed and executed the AWS → GCP migration, moved Fargate workloads to GKE standard node pools, and secured an MSP reseller discount
- Completed the full service migration via the 3-phase strategy — minimized downtime with resource-copy migration, and ran the final DNS cutover after announcing an approximately 2-hour maintenance window
- Codified the entire GCP infrastructure as Terraform IaC with 100% deployment history traceable through Git commits
- Eliminated GCP service-account key management overhead and key-leak risk for GitHub Actions by adopting Workload Identity Federation

---

Project 2: Game Data Collection Automation System

Contribution 100% (Python script development & operations) 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, costing analysts 2-3 hours daily and causing frequent data gaps from human error.

Approach

- **Multi-source ingestion into BigQuery** - MongoDB via a Dataflow Flex Template ETL, CloudSQL via BigQuery's direct connection, Google Analytics · Dune via their own APIs (Dune with a separate API key), all normalized into a consistent index schema

- **Auto-refreshed analyst-facing Google Sheets** – Python Cloud Functions push BigQuery query results into Google Sheets via the Sheets API, triggered by Cloud Scheduler on Daily (previous-day) and Monthly (previous-month) cadences
- **Full infrastructure as Terraform IaC** – BigQuery datasets, Cloud Functions, schedulers, Cloud Storage, Artifact Registry, IAM and service accounts all managed as code

Troubleshooting

- The CloudSQL → BigQuery direct connection (federated query) failed on region mismatch and missing connection-SA permissions.
  Resolved by creating the connection in the same region and granting the connection SA the Cloud SQL Client role
- Dune API's async execute → poll → fetch flow plus rate limits caused missing results / timeouts.
  Guaranteed result retrieval via execution-status polling with backoff and moved the API key into Secret Manager
- Daily re-runs re-loaded duplicate rows into BigQuery (non-idempotent).
  Applied MERGE / ROW_NUMBER() dedup logic in the periodically-run dedup Cloud Function to make loads idempotent, keeping zero gaps/duplicates
- Loading BigQuery results into Google Sheets failed on the 10M-cell-per-sheet limit, the write quota, and the per-minute request quota (429).
  Worked around the cell limit with chunked values.batchUpdate writes and sheet splitting, and stayed within quota via call pacing/throttling, batching, and exponential backoff

Results

- 95% data collection automation (eliminated 2-3h daily manual work), 0% data gap rate from human error
- Terraform IaC for the full infra enabled rebuilding the same pipeline in another GCP project within 30 minutes, with 100% change history in Git

---

Project 3: GPU Infrastructure Standardization & Scale-out for AI Workloads (Windows → Linux · Automated Setup · On-prem / GCP / External IDC)

Contribution 100% (Infra design, build & operations) 6 months (2024.02 ~ 2024.07)

Problem

The in-house GPU server ran Windows, so engineers hand-configured the runtime for every Stable Diffusion-family image-generation run. Matching driver, CUDA and framework versions by hand made the environment hard to reproduce — the hardware existed, but getting to work took time.

Approach

- **Windows → Linux standardization** – the in-house GPU server moved to Ubuntu 22.04, with a setup script installing and verifying the driver stack (nvidia-driver-535 · CUDA 11.5 · cuDNN 8.6) and nvidia-docker so whoever runs it gets the same environment; the image-generation workloads (Stable Diffusion WebUI · kohya_ss for LoRA training · ComfyUI) sit on top as a standard configuration
- **Cloud scale-out on GCP** – a VM with two NVIDIA A100 40GB cards (`a2-highgpu-2g`) provisioned as Terraform IaC, cleared through per-region/zone A100 availability checks and a GPU quota increase, then attached to the earlier migration's Shared VPC subnet so the network standard held
- **20 more servers at an external IDC, card tier split by workload** – 20 GPU servers built with the same setup script.
  Training needs VRAM and a multi-GPU interconnect (NVLink) because the model and batch do not fit on one card, while image-generation inference is independent per request and scales with machine count — so training and large-batch work run on the GCP A100s and bulk inference on consumer RTX 40-series servers at the IDC
- **Hardening alongside the publishers** – a new publishing deal kept the demand coming, and this GPU environment went on being hardened together with the publisher side right up to my departure

Results

- Replaced hand-configuration on Windows with Linux plus a setup script — the same script then provisioned all 20 external-IDC servers, turning a one-box procedure into the standard build path for the whole GPU fleet
- Managed the GPU server as Terraform code rather than a hand-built box, standardizing rebuild and change tracking — folded into the existing GCP IaC repository
- Scaled GPU capacity from a single in-house box to 10 on-prem boxes plus GCP A100 plus 20 servers at an external IDC — card tier split by workload (multi-GPU training vs per-request inference)

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and provided technical support.
Designed and built private cloud infrastructure for financial and public sector clients, and led training for client operations teams.

Project 1: Customized PaaS Solution for Clients

Contribution 100% (Infra design & Kubernetes cluster build) 11 months (2022.04 ~ 2023.03)

Problem

Each client had different environment and server spec requirements, making standardized solutions insufficient.

Approach

- **HA cluster design** – Kubernetes clusters designed in HA (High Availability) configuration to match each client's requirements, with a virtual machine management environment built on OpenStack
- **Unified storage** – block, file and object storage provided together through Ceph

Troubleshooting

- **Certificate expiry cycle** – cluster certificates expired on a one-year cycle, carrying a service-outage risk and a renewal job that returned every year at roughly 5 hours per client.
  Having analysed the known bug where kubeadm's built-in `certs renew` left some certificates unrenewed on v1.15~1.16, I established a version-independent script (update-kube-cert) that regenerates the certificates with a ten-year validity and restarts the components, and rolled it out across every client cluster incident-free

Results

- Built and delivered the HA-configured PaaS solution to 5 clients and ran it stably
- Ansible-based server provisioning and configuration management automation reducing per-client deployment time and achieving zero configuration drift
- Extended the certificate expiry cycle from one year to ten, eliminating the expiry-outage risk and cutting roughly 50 hours of annual operational effort across the 10 client sites then in operation

Project 2: Kubernetes Operations Training Program for DP

Contribution 100% (Training design & delivery) 6 months (2022.06 ~ 2022.11)

Problem

DP's operations team lacked Kubernetes and container-based infrastructure experience, anticipating difficulties in self-operation after PaaS solution adoption.

Approach

- **Step-by-step curriculum** – a training curriculum from Kubernetes basics through cluster operations and troubleshooting, run practice-first on hands-on lab environments
- **Parallel PaaS-stack training** – OpenStack and Ceph storage operations taught alongside, securing operational capability across the full PaaS stack

Results

- Operated the curriculum with an average of 10 attendees per session and 4 hours per session, establishing DP's autonomous Kubernetes operations system
- Reduced client technical inquiries post-training, achieved self-incident response capability
- Standardized training materials reused for subsequent client trainings